Conception et Développement d'un Framework de Sécurité Distribué Basé sur des Microservices, le Federated Learning et des Algorithmes Quantum-Inspired pour la Protection des Réseaux IoT Dynamiques

L'évolution fulgurante de l'Internet des Objets (IoT) a transformé radicalement les infrastructures numériques contemporaines, intégrant des milliards d'appareils connectés dans des secteurs critiques tels que la santé connectée, les villes intelligentes et l'automatisation industrielle.[1] Cependant, cette expansion sans précédent a engendré une surface d'attaque massive, rendant les paradigmes de sécurité traditionnels obsolètes. Les systèmes de détection d'intrusions (IDS) classiques, souvent conçus comme des entités monolithiques et centralisées, se heurtent aujourd'hui à des obstacles insurmontables dans des environnements dynamiques : la sensibilité des données interdit leur centralisation massive pour des raisons de confidentialité, l'hétérogénéité des terminaux complique le déploiement de politiques uniformes, et l'instabilité des connexions réseau nuit à la réactivité nécessaire face aux menaces émergentes comme les ransomwares.[1]

Pour surmonter ces défis, l'ingénierie moderne s'oriente vers la conception de frameworks distribués et modulaires. L'intégration de l'apprentissage fédéré (Federated Learning - FL) constitue une avancée majeure, permettant d'entraîner des modèles de détection sur des données décentralisées sans jamais exposer les informations brutes, garantissant ainsi la conformité avec des réglementations strictes comme le RGPD.[1] Parallèlement, l'adoption d'une architecture microservices offre la flexibilité nécessaire pour gérer la complexité de ces réseaux hétérogènes. En décomposant l'IDS en services indépendants et faiblement couplés, il devient possible d'adapter dynamiquement la sécurité à la topologie du réseau, de faciliter la scalabilité et d'améliorer la tolérance aux pannes.[3] Enfin, l'introduction d'algorithmes inspirés du calcul quantique (Quantum-Inspired) permet de doter ces systèmes d'une puissance d'optimisation supérieure, capable de traiter des espaces de recherche multidimensionnels pour la sélection de caractéristiques et la détection d'anomalies complexes, même sur du matériel classique actuel.[1]

Analyse Conceptuelle de la Problématique IoT et des

IDS Traditionnels

Les réseaux IoT modernes sont intrinsèquement instables et diversifiés. Un système de sécurité robuste doit impérativement prendre en compte la nature intermittente des connexions et la disparité des ressources de calcul entre un simple capteur de température et une passerelle intelligente.[1] Les IDS traditionnels souffrent d'une latence élevée due aux allers-retours constants vers le cloud, ce qui est inacceptable pour la détection de cyberattaques rapides telles que les ransomwares, qui peuvent chiffrer des milliers de fichiers en quelques secondes.[1]

L'approche monolithique limite également la capacité d'évolution. Si un nouveau protocole de communication est introduit ou si une nouvelle méthode de détection est développée, l'intégralité du système doit souvent être mise à jour et redéployée. Dans un contexte microservices, seul le service concerné est modifié, ce qui réduit considérablement les risques de régression et les temps d'arrêt.[6] La problématique centrale peut donc être reformulée comme suit : comment concevoir une infrastructure de sécurité qui soit à la fois modulaire pour suivre l'évolution technologique, décentralisée pour préserver la vie privée, et intelligente pour contrer des menaces sophistiquées par le biais de l'optimisation quantique?.[3]

Caractéristique	**IDS Monolithique Classique**	**Framework Proposé (FL + Microservices)**
Gestion des données	Centralisée (Risque de fuite élevé)	Localisée (Vie privée préservée via FL)
Architecture	Rigide, couplage fort	Modulaire, microservices indépendants
Adaptabilité	Faible face aux réseaux hétérogènes	Haute grâce à la conteneurisation
Optimisation	Heuristiques classiques	Algorithmes Quantum-Inspired (QGA)
Résilience	Point de défaillance unique	Distribuée, tolérance aux pannes

Architecture Globale de l'Écosystème Microservices

L'architecture du framework repose sur la distribution de l'intelligence à travers trois couches

fonctionnelles : la couche de perception (Edge), la couche de coordination (Fog) et la couche d'agrégation globale (Cloud).[9] Chaque couche héberge des microservices spécialisés qui communiquent via des protocoles légers et sécurisés.

La Couche Edge et le Micro-Agent de Sécurité

Au niveau des terminaux IoT ou des nœuds de proximité, nous déployons un micro-agent composé de plusieurs services essentiels. Cette approche permet de transformer chaque appareil en un capteur de menace proactif. Le service de surveillance des événements (FileEventService) est chargé de monitorer en temps réel les modifications du système de fichiers, en se concentrant particulièrement sur les fichiers exécutables (.exe,.dll) qui servent souvent de vecteurs d'attaque.[1] Dès qu'une modification suspecte est détectée, le service d'extraction de caractéristiques (ExtractFeaturesService) entre en action. Ce service transforme les métadonnées brutes (taille de débogage, adresses Bitcoin potentielles, entropie du fichier) en vecteurs de caractéristiques exploitables par les modèles d'apprentissage automatique.[1]

L'aspect critique de cette couche réside dans le service de prédiction locale (LocalPredictService). Ce service utilise un modèle de forêt aléatoire (Random Forest) ou un réseau de neurones léger pour classer les activités comme "bénignes" ou "suspectes".[1] En cas de détection positive, le service de réponse (ResponseService) peut immédiatement isoler l'appareil du réseau en modifiant les règles du pare-feu local, empêchant ainsi la propagation latérale de l'infection vers d'autres terminaux du réseau domestique.[1]

La Couche Fog et l'Optimisation Quantum-Inspired

Le niveau intermédiaire, ou Fog, joue le rôle de pont intelligent entre les terminaux Edge et le serveur d'agrégation Cloud. C'est ici que sont placés les microservices les plus gourmands en ressources de calcul, notamment le service d'optimisation basé sur l'Algorithme Génétique Quantique (QGA).[1] Ce service a pour mission de sélectionner le sous-ensemble de caractéristiques le plus pertinent pour la détection d'intrusions, réduisant ainsi la dimensionnalité des données et augmentant la vitesse d'entraînement des modèles locaux sur les terminaux Edge.[1]

L'algorithme QGA utilise une représentation probabiliste basée sur des qubits pour explorer simultanément plusieurs configurations de paramètres. Un individu est représenté par une chaîne de qubits $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, où $|\alpha|^2$ est la probabilité que la caractéristique ne soit pas sélectionnée.[7] Cette superposition permet une recherche globale bien plus efficace que les algorithmes génétiques classiques, évitant ainsi les optima locaux fréquents dans les jeux de données IoT complexes.[1] De plus, le service de gestion des ressources (QIARMService) utilise la superposition quantique pour allouer dynamiquement les tâches de calcul entre les nœuds Fog disponibles, garantissant une efficacité énergétique accrue de

l'ordre de 20%.[1]

Microservice	Fonction Principale	Emplacement	Impact sur la Sécurité
Ingestion & Monitoring	Capture de trafic et logs	Edge	Détection en temps réel
Extraction de Caractéristiques	Analyse de fichiers et métadonnées	Edge	Identification des patterns
Optimisation QGA	Sélection de caractéristiques	Fog	Réduction de la complexité
Entraînement Local	Apprentissage sur données privées	Edge/Fog	Confidentialité garantie
Agrégateur Global	Fusion des modèles via FedAvg	Cloud	Intelligence collaborative
Orchestrateur de Réponse	Isolation et remédiation	Fog	Confinement des menaces

Architecture Logique et Flux de Communication

L'architecture finale peut être visualisée comme un écosystème réactif orchestré par des événements. Le flux logique commence par l'initialisation du modèle global au niveau du serveur Cloud. Ce modèle est ensuite distribué aux terminaux Edge via un service de passerelle de paramètres sécurisé.[5]

Le Cycle d'Apprentissage Fédéré

Le processus suit un cycle itératif rigoureux. Chaque terminal Edge capture localement des données de trafic. Au lieu d'envoyer ces données au serveur, le service d'entraînement local (LocalTrainingService) calcule les mises à jour des poids du modèle en utilisant les caractéristiques optimisées par le service QGA.[1] Ces mises à jour, souvent cryptées par un mécanisme de chiffrement homomorphe pour prévenir toute fuite d'information, sont transmises au serveur Cloud.[1]

L'agrégateur global utilise l'algorithme FedAvg pour combiner les contributions de tous les

appareils. La formule de mise à jour globale s'énonce comme suit :

$$W_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_{k,t}$$

où $w_{k,t}$ représente les poids locaux du client k à l'itération t, et n_k le nombre d'échantillons de données locales.[5] Cette intelligence collective est ensuite redistribuée aux appareils, leur permettant de bénéficier de la connaissance acquise sur l'ensemble du réseau tout en gardant leurs propres données strictement privées.[1]

Intégration des Principes Quantiques dans l'Architecture

Le concept "Quantum-Inspired" ne se limite pas à l'optimisation des paramètres. Il infuse également la structure de représentation des données. En ingénierie, nous introduisons la notion de couches neuronales "sensibles à l'intrication" (Entanglement-Aware). Ces couches, bien qu'exécutées sur du matériel classique, modélisent les interdépendances complexes entre les caractéristiques de sécurité, simulant ainsi les corrélations non-locales observées en physique quantique.[1] Cela s'avère particulièrement efficace pour détecter des attaques par déni de service distribué (DDoS) ou des mouvements latéraux de ransomwares, où les anomalies sont dispersées à travers plusieurs dimensions du trafic réseau.[1]

L'architecture logique prévoit également des points d'intégration pour la Distribution de Clés Quantiques (QKD), notamment pour sécuriser les canaux de communication entre les serveurs Fog et le Cloud. Bien que la mise en œuvre actuelle puisse simuler ces protocoles, l'infrastructure est conçue pour être "future-proof", permettant le branchement direct de modules QKD matériels dès qu'ils deviennent accessibles dans les réseaux 6G.[1]

Structure de Projet et Standards de Développement (Monorepo)

Pour assurer une gestion efficace de ce système distribué, une structure de dépôt (Repository) de type **Monorepo** est préconisée. Cette approche facilite la gestion des dépendances communes, assure la cohérence des contrats d'API entre les services et simplifie les processus d'intégration et de déploiement continus (CI/CD).[15]

Organisation Recommandée du Dépôt Git

La structure est conçue pour être modulaire et scalable horizontalement, permettant l'ajout de nouveaux clients ou de nouveaux services de détection sans reconfigurer l'ensemble de la pipeline.

/security-framework-root ├── /apps (Applications finales) │ └── /ids-dashboard (Tkinter/Flask pour la visualisation NOC) [1] ├── /services (Microservices indépendants) │ ├── /event-monitor (Service de capture Edge - Python/Scapy) [1] │ ├── /feature-engine (Service QGA - Fog layer) [1] │ ├── /fl-client (Agent FL local - PyTorch/Flower) [13] │ ├── /fl-server (Agrégateur Cloud - gRPC) [5] │ └── /response-handler (Service de confinement - Scripts OS) [1] ├── /packages (Bibliothèques partagées) │ ├── /quantum-lib (Implémentations de QGA et Q-PCA) [11] │ ├── /common-dto (Objets de transfert de données partagés) [17] │ └── /crypto-utils (Chiffrement homomorphe et TLS) [14] ├── /deploy (Configuration infrastructure) │ ├── docker-compose.yml (Orchestration locale) [18] │ └── /kubernetes (Manifestes de déploiement cloud/fog) [8] ├── /data (Jeux de données de test - anomaly_data.csv) [1] └── pyproject.toml (Gestion des dépendances globales) [13]

L'utilisation de Docker est impérative pour encapsuler chaque service, garantissant que l'environnement d'exécution reste identique, qu'il soit déployé sur une passerelle IoT ARM64 ou sur un serveur cloud x86.[8] Le protocole gRPC est recommandé pour la communication inter-services en raison de sa performance et de son typage fort via Protocol Buffers, ce qui est crucial pour la transmission efficace des poids de modèles volumineux.[16]

Plan du Mémoire de Projet de Fin d'Études (PFE)

Le mémoire doit refléter la rigueur scientifique de l'approche quantum-inspired tout en démontrant une maîtrise technique de l'architecture microservices.

Chapitre 1 : Introduction et Contexte de l'IoT

Ce chapitre pose les bases de l'étude, analysant l'expansion du marché de l'IoT (estimé à plus de 20 milliards de dollars d'ici 2025) et la montée des menaces cybernétiques.[1] Il expose les limites des IDS monolithiques face à la décentralisation des données et définit les objectifs de modularité et de robustesse quantum-inspired.[3]

Chapitre 2 : État de l'Art : IA Distribuée et Calcul Quantique

Une revue critique des technologies existantes est menée. On y compare les différentes topologies de FL (centralisée, hiérarchique, décentralisée) et on introduit les concepts de physique quantique appliqués à l'informatique : superposition, intrication et leur application dans les métaheuristiques comme le QGA.[1]

Chapitre 3 : Conception du Framework Microservices

Ce chapitre détaille l'architecture logique du système. Il décrit la décomposition en microservices, les protocoles de communication (gRPC/TCP) et le choix de la plateforme d'orchestration. Une attention particulière est portée à la conception des agents coopératifs

(FileEventAgent, PredictAgent, etc.).[1]

Chapitre 4 : Intégration de l'Apprentissage Fédéré et de l'Optimisation Quantique

C'est le cœur théorique du projet. Il formalise l'algorithme d'agrégation FedAvg et détaille l'implémentation de l'algorithme génétique quantique pour la sélection de caractéristiques. On y explique mathématiquement comment la rotation des portes quantiques permet une convergence plus rapide vers le modèle optimal.[5]

Chapitre 5 : Réalisation Technique et Simulation

Ce chapitre décrit l'environnement de test (simulation d'un réseau Wi-Fi domestique sur Azure avec plusieurs VMs Windows/Linux). Il présente la pile logicielle utilisée (Python 3.10, PyTorch, Flower, Docker) et le processus de collecte des données (anomaly_data.csv).[1]

Chapitre 6 : Évaluation des Performances et Analyse des Résultats

Une analyse rigoureuse des KPIs est présentée. On compare le framework proposé avec des modèles centralisés classiques. Les tests de robustesse sous déconnexion et l'impact de l'optimisation QGA sur la précision de détection (jusqu'à 98%) sont discutés.[4]

Chapitre 7 : Conclusion et Perspectives

Le mémoire se conclut sur les apports du projet, notamment la scalabilité et la protection de la vie privée. Il ouvre des perspectives sur l'intégration future avec la 6G et l'usage de blockchain pour sécuriser les historiques de mise à jour des modèles.[1]

Scénarios de Test et Expérimentation

Pour valider l'efficacité du système, nous avons défini plusieurs scénarios simulant les conditions réelles d'un réseau domestique hétérogène. L'objectif est de mesurer non seulement la précision de détection, mais aussi la capacité du système à fonctionner dans des environnements dégradés.

Scénario 1 : Base de Référence et Convergence du Modèle

Dans ce scénario, trois nœuds Edge stables reçoivent des données distribuées de manière identique (IID). On observe la vitesse de convergence du modèle global. Les résultats attendus montrent une amélioration constante de la précision, atteignant un plateau autour de la 10ème ronde de communication avec une précision de 91.2%.[1]

Scénario 2 : Résilience face à l'Hétérogénéité des Données (Non-IID)

C'est le test le plus critique pour un système FL. Chaque appareil voit des types d'attaques

différents ou possède des volumes de données très disparates (par exemple, une caméra de sécurité génère plus de logs qu'une ampoule intelligente). On évalue si l'agrégateur parvient à maintenir une précision élevée sans être biaisé par un nœud dominant.[13]

Scénario 3 : Robustesse sous Déconnexion Dynamique

On simule des déconnexions aléatoires de nœuds pendant la phase d'entraînement local. Le framework doit être capable de poursuivre l'agrégation globale en utilisant un quorum minimal de participants. Ce test mesure la tolérance aux pannes induite par l'architecture microservices et l'orchestrateur Fog.[1]

Scénario 4 : Détection de Ransomware et Temps de Réponse

On introduit un échantillon de ransomware actif sur un nœud Edge. On mesure le temps nécessaire pour que le FileEventAgent détecte l'activité, que le LocalPredictService confirme la menace et que le ResponseService isole la machine. L'objectif est d'atteindre un temps de réaction inférieur à 50ms pour minimiser les pertes de données.[1]

Métriques de Performance Clés (KPIs)

Métrique	Définition / Objectif	Résultat Cible
Précision (Accuracy)	Ratio de détections correctes sur le total	> 95%
F1-Score	Équilibre entre précision et rappel	> 0.91
Latence de Détection	Temps entre l'intrusion et l'alerte	< 150 ms
Vitesse de Convergence	Nombre de rondes pour atteindre 90% d'acc.	< 10 rondes
Consommation Énergie	Énergie économisée via optimisation Fog	- 20%
Overhead Communication	Réduction du trafic par rapport au centralisé	- 35%

L'analyse de ces métriques permet de confirmer que le framework ne sacrifie pas la

performance pour la modularité. En effet, l'utilisation de l'algorithme QGA pour la sélection de caractéristiques permet souvent d'augmenter la précision de 4 à 5% par rapport aux méthodes classiques tout en réduisant le temps d'entraînement local.[1]

Vue Globale du Produit Final : Le Framework QFL-IDS

Le produit final se présente comme une infrastructure de sécurité "zéro confiance" (Zero Trust) conçue pour l'ère de l'intelligence distribuée. Il ne s'agit pas seulement d'un logiciel de détection, mais d'un véritable écosystème réactif.[3]

Interface de Visualisation et Centre d'Opérations (NOC)

L'administrateur du réseau dispose d'un tableau de bord centralisé qui visualise l'état de santé de tous les terminaux IoT. Grâce à l'architecture microservices, le statut de chaque appareil est mis à jour en temps réel via des webhooks. Un système de code couleur intuitif permet une gestion rapide :

- **Vert :** Fonctionnement normal, entraînement local actif.[1]
- **Jaune :** Pas de mise à jour récente (déconnexion potentielle) ou divergence du modèle local détectée.[1]
- **Rouge :** Intrusion confirmée par le service de prédiction. La machine est marquée comme isolée et une empreinte de l'attaque est partagée avec les autres nœuds.[1]

Intelligence Collaborative et Évolution Continue

La force du produit réside dans sa capacité d'apprentissage continu. Chaque fois qu'une nouvelle menace est identifiée sur un terminal, les caractéristiques distillées de cette menace sont intégrées dans le cycle de mise à jour globale. Ainsi, l'ensemble du framework devient plus résilient contre cette menace spécifique en l'espace de quelques minutes, sans que les données privées des utilisateurs n'aient jamais quitté leur domicile.[1]

L'intégration des algorithmes quantum-inspired garantit que le framework reste performant même face à une augmentation exponentielle du nombre d'appareils connectés. En optimisant finement l'utilisation des ressources via QIARM et en maintenant une sélection de caractéristiques légère via QGA, le système offre une scalabilité linéaire, permettant de protéger des réseaux allant d'un simple Wi-Fi domestique à des infrastructures industrielles complexes de plus de 10 000 nœuds.[1] Ce framework représente donc une pierre angulaire pour l'évolution durable et sécurisée des écosystèmes IoT modernes, conciliant performance technologique et impératifs éthiques de confidentialité.[3]

Sources des citations

1. TSP_CMES_60973.pdf
2. [2510.20852] FedMicro-IDA: A Federated Learning and Microservices-based

Framework for IoT Data Analytics - arXiv, consulté le février 12, 2026, https://arxiv.org/abs/2510.20852
3. (PDF) FedMicro-IDA: A Federated Learning and Microservices-based Framework for IoT Data Analytics - ResearchGate, consulté le février 12, 2026, https://www.researchgate.net/publication/396924473_FedMicro-IDA_A_Federated_Learning_and_Microservices-based_Framework_for_IoT_Data_Analytics
4. Federated Learning-Driven Cybersecurity Framework for IoT Networks with Privacy- Preserving and Real-Time Threat Detection Capabilities - arXiv, consulté le février 12, 2026, https://arxiv.org/pdf/2502.10599
5. (PDF) Federated Learning-Driven Cybersecurity Framework for IoT Networks with Privacy-Preserving and Real-Time Threat Detection Capabilities - ResearchGate, consulté le février 12, 2026, https://www.researchgate.net/publication/389090434_Federated_Learning-Driven_Cybersecurity_Framework_for_IoT_Networks_with_Privacy-Preserving_and_Real-Time_Threat_Detection_Capabilities
6. Exploring the Potential of Microservices in Internet of Things: A Systematic Review of Security and Prospects - MDPI, consulté le février 12, 2026, https://www.mdpi.com/1424-8220/24/20/6771
7. A Novel Quantum Epigenetic Algorithm for Adaptive Cybersecurity Threat Detection - MDPI, consulté le février 12, 2026, https://www.mdpi.com/2673-2688/6/8/165
8. Microservices-Oriented Middleware for Federated IoT Data Processing: Integrating Federated Learning Principles into Middleware for Distributed IoT Data Handling | Request PDF - ResearchGate, consulté le février 12, 2026, https://www.researchgate.net/publication/387971202_Microservices-Oriented_Middleware_for_Federated_IoT_Data_Processing_Integrating_Federated_Learning_Principles_into_Middleware_for_Distributed_IoT_Data_Handling
9. AI-driven Microservices for Scalable IoT Architectures: A Design and Implementation Perspective - ResearchGate, consulté le février 12, 2026, https://www.researchgate.net/publication/397926314_AI-driven_Microservices_for_Scalable_IoT_Architectures_A_Design_and_Implementation_Perspective
10. Quantum-Inspired Feature Engineering for Intrusion Detection Systems - ResearchGate, consulté le février 12, 2026, https://www.researchgate.net/publication/392455300_Quantum-Inspired_Feature_Engineering_for_Intrusion_Detection_Systems
11. Federated Learning, Part 2: Implementation with the Flower Framework, consulté le février 12, 2026, https://towardsdatascience.com/federated-learning-part-2-implementation-with-the-flower-framework-%F0%9F%8C%BC/
12. Federated Learning Framework for IoT Security | Enrique Tomás Martínez Beltrán, consulté le février 12, 2026, https://enriquetomasmb.com/project/federated-learning-framework-for-iot-security/
13. How to choose the right code repository for microservices - Harness, consulté le février 12, 2026,

https://www.harness.io/harness-devops-academy/best-code-repository-for-microservices
14. Federated Learning with gRPC and Docker - GitHub, consulté le février 12, 2026, https://github.com/mayankshah1607/federated-learning-with-grpc-docker
15. How to structure microservices in your repository - Software Engineering Stack Exchange, consulté le février 12, 2026, https://softwareengineering.stackexchange.com/questions/386066/how-to-structure-microservices-in-your-repository
16. A project structure when using microservices and docker : r/Python - Reddit, consulté le février 12, 2026, https://www.reddit.com/r/Python/comments/3qucks/a_project_structure_when_using_microservices_and/
17. Micro-FL: A Fault-Tolerant Scalable Microservice-Based Platform for Federated Learning, consulté le février 12, 2026, https://www.mdpi.com/1999-5903/16/3/70
18. AI-Enabled IoT Intrusion Detection: Unified Conceptual Framework and Research Roadmap, consulté le février 12, 2026, https://www.mdpi.com/2504-4990/7/4/115
19. Quantum-Inspired Algorithms and Perspectives for Optimization - MDPI, consulté le février 12, 2026, https://www.mdpi.com/2079-9292/14/14/2839